Search Algorithms at a Glance

EECS 492

Winter 2026

Expand

```
function EXPAND(problem, node) yields nodes
    s ← node.STATE
    for each action in problem.ACTIONS(s) do
        s' ← problem.RESULT(s, action)
        cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
        yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

Your current state

All the things you could do in the state

Where you could end up

How expensive it would be to get there

generates a sequence of values, each time yield is encountered

*check that the root isn't a goal

Breadth-First Search

```
function BREADTH-FIRST-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a first-in-first-out queue, with node as an element*
    reached ← a set of states, with one entry with value node.STATE

    if problem.IS-GOAL(node.STATE) then return node
    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        for each child in EXPAND(problem, node) do
            if problem.IS-GOAL(child.STATE) then return child
            s ← child.STATE
            if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
                reached[s] ← child
                add child to frontier

    return failure
```

Complete: Yes
(goal at finite depth & b ≠∞)

Optimal: Yes
(if all actions are equal cost)

Time Complexity: $O(b^d)$
Space Complexity: $O(b^d)$

Goal Test: At Generation

Early **goal test** – check if a node is a solution as soon as it is *generated*, rather than waiting until it is popped off the queue.

Reached: stores states
Frontier: first-in-first-out queue

Depth-First Search

Note: We assume there are no cycles and do not save a *reached* set (saves memory!)

```
function DEPTH-FIRST-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a last-in first-out queue, with node as an element

    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            add child to frontier
    return failure
```

In iterative deepening search, you check the depth level here

Late goal test – checking whether a node is a solution node when it is popped off the queue

Complete: No, may run into an infinite cycle
Optimal: No

(m – max depth)
Time Complexity: $O(b^m)$
Space Complexity: O(bm)

Goal Test: At Expansion
(Note that you can goal test at either expansion or generation – expansion is just the convention we follow)

Reached: not necessary
Frontier: last-in-first-out queue

Uniform Cost Search

```
function UNIFORM-COST-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a priority queue ordered by f, with node as an element
    reached ← a lookup table, with one entry with key problem.INITIAL and value node

    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
                reached[s] ← child
                add child to frontier
    return failure
```

Complete: Yes
(if goal at finite depth & b ≠∞, path cost is greater than ε)
Optimal: Yes
(if step-costs are non-negative)

Time Complexity: $O(b^{1+\lceil C^*/\varepsilon \rceil})$
Space Complexity: $O(b^{1+\lceil C^*/\varepsilon \rceil})$
[C^* = the cost of the optimal solution
ε = the minimum step-cost]

Goal Test: At Expansion

Reached: stores nodes
Frontier: priority queue, path-cost

DFS Variants

- Depth Limited Search – limit DFS to a max depth of L
  - Complete: Only if goal depth <= L
  - Optimal: No
  - Time Complexity: $O(b^L)$
  - Space Complexity: O(bL)

- Iterative Deepening DFS – Start at L=0 and repeat Depth Limited search increasing L each time till you find a goal
  - Complete: Yes, if goal at finite depth & $b \neq \infty$
  - Optimal: Yes if all actions cost the same
  - Time Complexity: $O(b^d)$
  - Space Complexity: O(bd)

Greedy Best-First-Search

```
function GREEDY-BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a priority queue ordered by h, with node as an element
    reached ← a lookup table, with one entry with key problem.INITIAL and value node

    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
                reached[s] ← child
                add child to frontier
    return failure
```

Complete:
Yes (if goal at finite depth & b ≠∞,)
Optimal: No

Time Complexity: Depends on h - $O(b^m)$ worst case
Space Complexity: Same as time

Goal Test: At Expansion

A* Search

```
function A*-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a priority queue ordered by h+g, with node as an element
    reached ← a lookup table, with one entry with key problem.INITIAL and value node

    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
                reached[s] ← child
                add child to frontier
    return failure
```

Complete:
Yes
(if goal at finite depth & b ≠∞, path cost is greater than ε)
Optimal: Yes (and optimally efficient) if you have a consistent & admissible heuristic

Time Complexity: depends on h, worst case - $O(b^m)$
Space Complexity: same as time

Goal Test: At Expansion